Comparison between lists and sets in Python

Feature	List	Set
Order of Elements	Maintains the order in which items are added	Does not maintain order (unordered)
Duplicates	Allows duplicates, e.g., [1, 1, 2, 3]	Automatically removes duplicates
Syntax	Defined with square brackets [], e.g., [1, 2]	Defined with curly braces {}, e.g., {1, 2}
Indexing	Supports indexing, e.g., my_list[0]	Does not support indexing
Common Use Cases	Use when order and duplicates matter	Use for unique elements, membership testing
Mutability	Mutable; can add, remove, or modify elements	Mutable; can add and remove but not modify
Adding Elements	Use .append() or .insert() to add items	Use .add() to add an item
Removing Elements	Use .remove() or .pop()	Use .remove() or .discard()
Iteration Speed	Slower than sets for membership testing	Faster for membership testing
Memory Usage	Requires more memory for duplicates	Uses less memory as duplicates are removed
Sorting	Can be sorted using .sort()	Must convert to list to sort
Example of Uniqueness	[1, 2, 2, 3] keeps all 2s	{1, 2, 2, 3} will only keep one 2
Supported Operations	Supports slicing, concatenation, and repetition	Supports set operations like union, intersection, and difference
Union of Two Collections	list1 + list2 (concatenation)	set1.union(set2)
Use in Functions	Ideal for ordered collections	Ideal for testing membership or uniqueness
Example Adding	my_list.append(4)	my_set.add(4)
Removing Duplicates	Cannot automatically remove duplicates	Removes duplicates automatically